1-Simple Factory (Resuelto por Joxe Echeveste)

Hay muchas vulnerabilidades en esta aplicación:

A. ¿Qué sucede si aparece un nuevo síntoma (por ejemplo, mareos)?

En el caso de que apareciese un nuevo síntoma; ese síntoma debería añadirse a alguno de los arrays en el método createSymptom en la clase Covid19Pacient; sea en el array de impact5, el de impact 3 o el de impact1; y a su vez, en uno de los arrays de los tipos generales de síntoma; sea digestivo, neuromuscular o respiratorio.

B. ¿Cómo se puede crear un nuevo síntoma sin cambiar las clases existentes (principio OCP)?

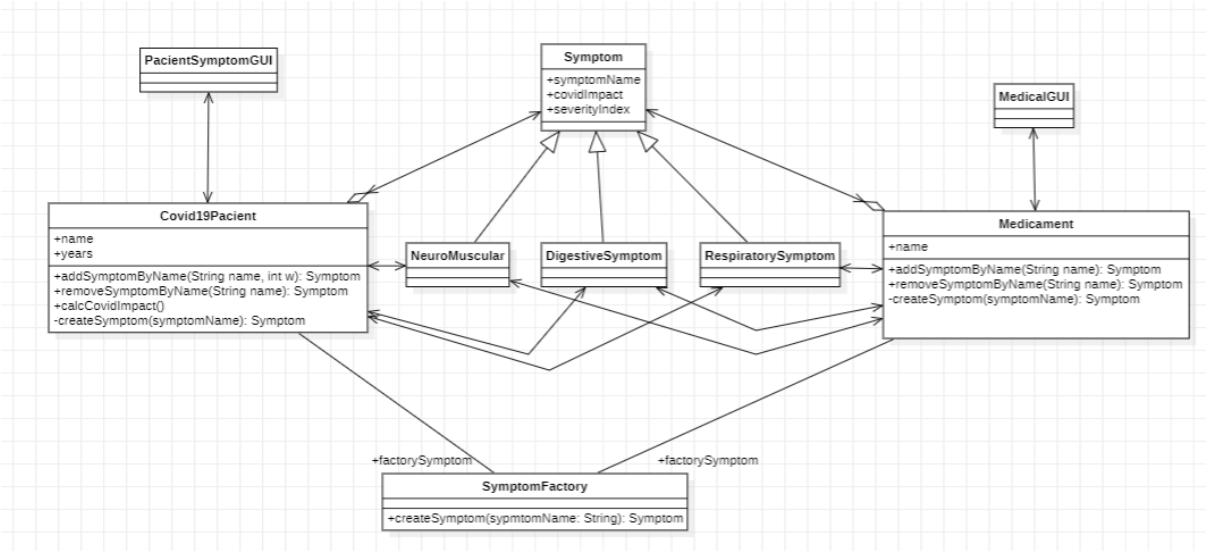
Para crear una clase Síntoma que contenga todos los síntomas; de modo que todos los cambios relacionados con los síntomas se hagan en la clase Síntoma; y Covid19Pacient y Medicament solo se ocupen de obtener los síntomas creados.

C. ¿Cuántas responsabilidades tienen las clases de Covid19Pacient y Medicament (principio SRP)?

La clase Covid19Pacient tiene 4 responsabilidades (addSymptomByName(...), removeSymptomByName(...), calcCovidImpact() y createSymptom(...)). La clase Medicament, en cambio, tiene 4 responsabilidades (addSymptomByName(...), removeSymptomByName(...) y createSymptom(...)).

Se pide:

1. Realiza un nuevo diseño de la aplicación (diagrama UML) aplicando el patrón Simple Factory para eliminar vulnerabilidades anteriores y mejorar el diseño en general. Describe con claridad los cambios realizados.



Hemos cogido el código repetido en los métodos createSymptom(...) de las clases Covid19Pacient y Medicament; y lo hemos metido dentro de la clase SymptomFactory; la cual tiene un método llamado

createSymptom(...). En las clases donde hemos quitado el código repetido hemos añadido una llamada al método createSymptom(...) de la clase SymptomFactory. Para ello, añadimos como atributo y parámetro en la clase constructora de cada clase un elemento de tipo SymptomFactory.

De esta manera, hacemos que sea la clase SymptomFactory la que se ocupe de crear síntomas y de gestionar la adición de síntomas nuevos; y Covid19Pacient y Medicament sólo se preocupan de obtener los objetos que devuelve SymptomFactory.

2. Implementa la aplicación y agrega el nuevo síntoma "mareos" asociado a un tipo de impacto 1.

Hecho, en la clase SymptomFactory.java en la carpeta factory

3. Cómo se puede adaptar la clase Factory, para que los objetos Symptom que utilicen las clases Covid19Pacient y Medicament sean únicos. Es decir, para cada síntoma sólo exista un objeto. (Si hay x síntomas en el sistema, haya únicamente x objetos Symptom)

Para adaptar la clase Factory de modo que los objetos de las clases que lo utilizan sean únicos, se podría añadir un contador por síntoma al inicio del factory; con el número de objetos Symptom posibles (En nuestro caso, 3 síntomas; entonces 3 contadores). Si la llamada al factory es en un objeto con contador 0, se devolverá una excepción reflejando que no quedan más objetos por crear; y en caso contrario se devolverá el objeto que se quiera crear; no sin antes haber decrementado su contador correspondiente, llegando así a 0.

2-Patrón Observer (Resuelto por Ane Miren Alvarez)

Se pide: cambia el programa principal para crear 2 pacientes Covid19Pacient con sus interfaces PacientSymptomGUI y PacientObserverGUI.

Hecho, en la clase Main.java en el paquete observer

3-Patrón Adapter (Resuelto por Joxe Echeveste)

Se pide:

• Añade el código necesario en la clase Covid19PacientTableModelAdapter y ejecuta la aplicación para comprobar que funciona correctamente.

Hecho, en la clase Covid19PacientTableModelAdapter.java en el paquete adapter2.

• Añade otro paciente con otro síntomas, y ejecuta la aplicación para que aparezcan los 2 pacientes son sus síntomas.

Comprobado, en la clase Main.java en el paquete adapter2.

• **(opcional)**. Cómo podrías añadir esta nueva pantalla al ejercicio anterior del observer, de forma que cada vez que se añada un nuevo síntoma a un paciente, se actualice la tabla.

Para que se actualice la tabla cada vez que se añade algo nuevo al paciente; debemos hacer que la tabla implemente un patrón observer.

Esto haría que la tabla; al observar un cambio en los síntomas del paciente, añada la información a la tabla.

Para ello, tenemos que hacer que, en la clase constructora de la tabla; se le añade un observador al paciente con la tabla que llama a la constructora. Cabe aclarar que debemos hacer que Covid19Pacient extienda la clase Observer para que pueda llamar al método addObserver(...) correctamente. Además, haremos que nuestro objeto con la instancia de Covid19PacientTableModelAdapter sea AbstractTableModel para poder actualizar la tabla. Para finalizar, añadiremos la función update que debemos implementar al implementar la interfaz Observer; y haremos que el update se ejecute cuando los datos de la tabla cambien; en este caso al añadir un síntoma nuevo.

4-Patrón Iterator y Adapter (Resuelto por Ane Miren Alvarez)

Se pide:

Crea un programa principal utilizando el método Sorting.sortedIterator que imprima los 5 síntomas que debe tener un paciente Covid19Pacient. Se imprimirá primero ordenando por symptomName y luego por severityIndex.

• Crea un paciente Covid19Pacient con cinco síntomas. La clase Covid19Pacient NO PUEDE CAMBIARSE NADA.

• Implementa las interfaces Comparator: una para la ordenación por symptomName, y otra para la ordenación según severityIndex.

• Crea el patrón adapter sobre la clase Covid19Pacient, implementando la interfaz InvertedIterator. Recuerda crear una constructora adecuada para enviarle la información del paciente.

• TAMPOCO SE PUEDE MODIFICAR Sorting.sortedIterator.

Hecho, en la clase